This will create the initial directory structure and code for a Rails application. It will also run bundle install to load initial gems. You're going to mostly stick with the initial gem set for the moment. I assume that you're already familiar with Rails core concepts; I won't spend a lot of time reexplaining them. If you're not familiar with Rails, *Agile Web Development with Rails [Rub13]* is still the gold standard for getting started.

You need to create your databases. For ease of setup and distribution, stick to the Rails default—SQLite.[1] Run the following in your terminal to create the database:

```
% rake db:create:all
% rake db:migrate
```

You need the db:migrate call even though you haven't actually created a database migration, because it sets up the schema.rb file that Rails uses to rebuild the test database. The test database is automatically maintained when the schema.rb file changes.

The Requirements

The most complex business logic you need to build concerns forecasting a project's progress. You want to be able to predict the end date of a project and determine whether that project is on schedule.

In other words, given a project and a set of tasks, some of which are done and some of which are not, use the rate at which tasks are being completed to estimate the project's end date. Also, compare that projected date to a deadline to determine if the project is on time.

This is a good example problem for TDD because, while it's not hard to have a sense of what the answer is, the best way to structure the code is not clear. TDD will help, guiding you toward reasonable code design.

Installing RSpec

Before you start testing, you'll need to load RSpec, your testing library.

To add RSpec to a Rails project, add the rspec-rails gem to your Gemfile:

```
group :development, :test do
  gem "rspec-rails", "~> 3.7.0"
end
```

1. http://www.sqlite.org